EX.NO: 01	DOWNLOADING AND INSTALLING HADOOP; UNDERSTANDING DIFFERENT HADOOP MODES. STARTUP SCRIPTS, CONFIGURATION FILES.
DATE:	

AIM:

To download, install Apache Hadoop and study different Hadoop modes, startup scripts and configuration files

REQUIREMENTS :

- OS: Linux (Ubuntu)
- Java (JDK 8 or 11)
- Internet connection

THEORY / DESCRIPTION

Apache Hadoop:

Apache Hadoop is an open-source framework used for storing and processing large amounts of data in a distributed environment. It allows data to be stored across multiple systems (nodes) and processed in parallel, improving performance and scalability.

Hadoop mainly consists of two core components:
HDFS (Hadoop Distributed File System) for distributed storage and **YARN (Yet Another Resource Negotiator)** for resource management and task execution. It is fault-tolerant, meaning data is replicated across nodes to prevent data loss.

Hadoop can run in different modes such as standalone mode, pseudo-distributed mode, and fully distributed mode. In fully distributed mode, multiple systems are connected to form a cluster.

Clusters can be:

- **Homogeneous cluster:** All nodes have similar configuration
- **Heterogeneous cluster:** Nodes have different configurations

Hadoop provides several built-in example programs to demonstrate distributed data processing, such as:

- **Pi** – Used to estimate the value of π using parallel computation

- **WordCount** – Counts the number of occurrences of each word in a dataset
- **Grep** – Searches for specific patterns in large datasets
- **Sort** – Sorts large volumes of data in distributed manner
- **TeraSort** – Performs large-scale sorting (benchmark program)
- **RandomTextWriter** – Generates random text data for testing
- **RandomWriter** – Generates random data (text/binary) for input

These programs help in understanding how Hadoop processes data using distributed computing.

In this experiment, Hadoop was installed and configured in a Linux system. The necessary configuration files were modified, and Hadoop services were started using startup scripts. These example programs were executed to verify the working of Hadoop and to understand its processing capabilities.

PROCEDURE :

STEP 1: Install Java

sudo apt update

sudo apt install openjdk-8-jdk -y

STEP1.1: Set JAVA_HOME:

nano ~/.bashrc

Add:

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

export PATH=$PATH:$JAVA_HOME/bin

Apply:

source ~/.bashrc

STEP 2: Create Hadoop User

sudo adduser hadoop

STEP 2.1: Give Sudo Access

sudo usermod -aG sudo hadoop

STEP 2.3: Switch to Hadoop User

su – Hadoop

STEP 3: Download Hadoop

Download from Apache Software Foundation official site

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

STEP 3: Extract Hadoop

tar -xvzf hadoop-3.3.6.tar.gz

mv hadoop-3.3.6 hadoop

STEP 4: Set Hadoop Environment Variables

nano ~/.bashrc

Add:

export HADOOP_HOME=~/hadoop

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

Apply:

source ~/.bashrc

STEP 5: Configure Hadoop

Go to config folder:

cd ~/hadoop/etc/Hadoop

open the core-site.xml

nano core-site.xml

```
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

- **Open the hdfs-site.xml**

nano hdfs-site.xml

```
<configuration>
 <property>
<name>dfs.replication</name>
<value>1</value>
</property>
```

</configuration>

- **Open the mapred-site.xml**

cp mapred-site.xml.template mapred-site.xml

nano mapred-site.xml

<configuration>

<property>

<name>mapreduce.framework.name</name>

<value>yarn</value>

</property>

</configuration>

- **Open the yarn-site.xml**

nano yarn-site.xml

<configuration>

<property>

<name>yarn.nodemanager.aux-services</name>

<value>mapreduce_shuffle</value>

</property>

</configuration>

STEP 6: Format NameNode

hdfs namenode -format

STEP 7: Start Hadoop

start-dfs.sh

start-yarn.sh

STEP 8: Verify Installation

Jps

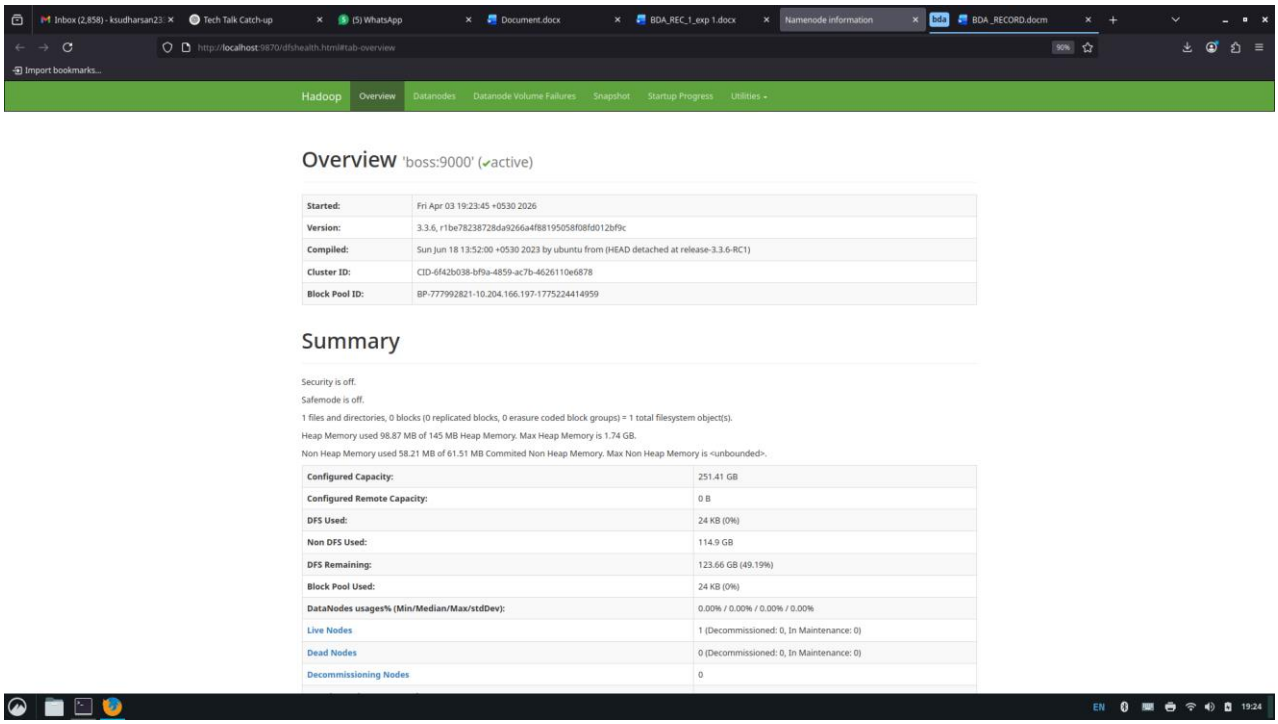
Expected:

- NameNode
- DataNode
- ResourceManager
- NodeManager

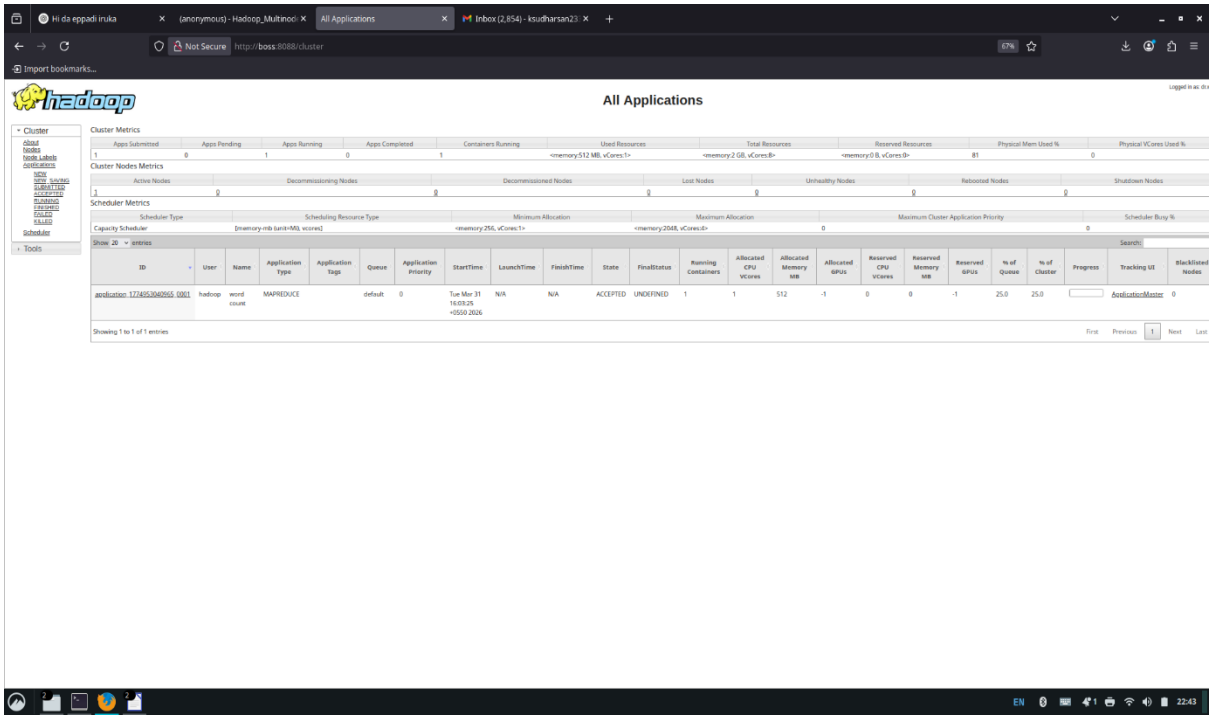
STEP 9: Web Interface

NameNode → http://localhost:9870

ResourceManager → http://localhost:8088



NameNode → http://localhost:9870



ResourceManager → http://localhost:8088

RESULT:

Downloading and installing hadoop; understanding different hadoop modes. Startup scripts, configuration files are successfully implemented.

EX.NO: 02	

DATE:	HADOOP IMPLEMENTATION OF FILE MANAGEMENT TASKS, SUCH AS ADDING FILES AND DIRECTORIES, RETRIEVING FILES AND DELETING FILES

AIM:

To perform file operations like creating directories, adding files, retrieving and deleting files in HDFS.

PROCEDURE :

STEP 1: Create Directory

Command:

hdfs dfs -mkdir /testdir

STEP 1.1 : Create Subdirectory

Command:

hdfs dfs -mkdir /testdir/subdir

S**TEP 1.2:** List Files

Command:

hdfs dfs -ls /

Ouput:

```
hadoop@boss:~$ hdfs dfs -mkdir /testdir
hadoop@boss:~$ hdfs dfs -mkdir /testdir/subdir
hadoop@boss:~$ hdfs dfs -ls /
Found 1 items

drwxr-xr-x   - hadoop supergroup          0 2026-04-03 19:57 /testdir
hadoop@boss:~$
```

STEP 2: Create Local File

Command:

nano file.txt

Add content:

Hello Hadoop

Save → CTRL + X → Y → Enter

STEP 2.1: Verify Local File

Command:

cat file.txt

Output:

hadoop@boss:~$ nano file.txt

hadoop@boss:~$ cat file.txt

hello hadoop

hadoop@boss:~$

STEP 3: Upload File to HDFS

Command:

hdfs dfs -put file.txt /testdir

STEP 3.1: List Files in HDFS

Command:

hdfs dfs -ls /testdir

Output :

hadoop@boss:~$ hdfs dfs -put file.txt /testdir

hadoop@boss:~$ hdfs dfs -ls /testdir

Found 2 items

-rw-r--r-- 1 hadoop supergroup 13 2026-04-03 20:16 /testdir/file.txt

drwxr-xr-x - hadoop supergroup 0 2026-04-03 19:57 /testdir/subdir

hadoop@boss:~$

STEP 3.2: View File from HDFS

Command:

hdfs dfs -cat /testdir/file.txt

Output:

Hello Hadoop

STEP 4: Download File (Retrieve)

Command :

hdfs dfs -get /testdir/file.txt

STEP 4.1: Delete File

Command :

hdfs dfs -rm /testdir/file.txt

Output:

hadoop@boss:~$ hdfs dfs -rm /testdir/file.txt

Deleted /testdir/file.txt

STEP 4.2: Delete Directory

Command:

hdfs dfs -rm -r /testdir

Output :

hadoop@boss:~$ hdfs dfs -rm -r /testdir

Deleted /testdir

hadoop@boss:~$

STEP 5: Verify (it does not show the list of files and folders)

Command :

hdfs dfs -ls /

Output:

hadoop@boss:~$ hdfs dfs -ls

hadoop@boss:~$

RESULT:

File management operations like create, upload, retrieve and delete were successfully performed in HDFS.

EX.NO: 03	IMPLEMENTATION OF MATRIX MULTIPLICATION USING HADOOP MAPREDUCE
DATE:	

AIM:

To implementation of matrix multiplication using hadoop mapreduce

PROCEDURE:

We assume that the input matrices are already stored in the Hadoop Distributed File System (HDFS) in a suitable format (such as CSV or TSV), where each row represents an element of the matrix. The matrices must be compatible for multiplication, i.e., the number of columns in the first matrix must be equal to the number of rows in the second matrix.

STEP 1: MAPPER

The Mapper reads each input record and processes the matrix elements. It identifies whether the element belongs to matrix A or matrix B and emits intermediate key-value pairs.

The key represents the position (row, column) of the resulting matrix element, and the value represents the corresponding matrix element required for multiplication.

STEP 2: REDUCER

The Reducer receives grouped key-value pairs from the Mapper phase. For each key, it performs the multiplication and summation of corresponding matrix elements to compute the final value of the result matrix element.

STEP 3: MAIN DRIVER

The Main Driver class is responsible for configuring and initializing the Hadoop job. It sets the Mapper and Reducer classes, defines input and output data types, and specifies the input and output paths in HDFS.

STEP 4: RUNNING THE JOB

The MapReduce program is compiled and packaged into a JAR file. The job is then submitted to Hadoop using the following command:

hadoop jar matrix.jar MatrixMultiplication /input /output

Ensure that the input and output paths are correctly specified in HDFS. If the output directory already exists, it must be deleted before running the job.

STEP 5: Create CSV file

nano matrix.csv

Paste this:

A,0,0,1

A,0,1,2

A,1,0,3

A,1,1,4

B,0,0,5

B,0,1,6

B,1,0,7

B,1,1,8

STEP 6: Verify file

cat matrix.csv

STEP 7: Create HDFS folder

hdfs dfs -mkdir /matrix

STEP 8: Upload CSV to HDFS

hdfs dfs -put matrix.csv /matrix

STEP 9: Check upload

hdfs dfs -ls /matrix

Ouput:

matrix.csv

PROGRAM :

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MatrixMultiplicationDriver {
   public static class MatrixMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{
      public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
         String line = value.toString();
         String[] parts = line.split(",");
         // parts: MatrixName,row,column,value
```

```
            String matrix = parts[0];
            int row = Integer.parseInt(parts[1]);
            int col = Integer.parseInt(parts[2]);
            int val = Integer.parseInt(parts[3]);
            // Key: row,column (result position)
            context.write(new Text(row + "," + col), new IntWritable(val));
        }
    }
    public static class MatrixReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Matrix Multiplication");
        job.setJarByClass(MatrixMultiplicationDriver.class);
        job.setMapperClass(MatrixMapper.class);
        job.setReducerClass(MatrixReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

STEP 10 : Compile the java program using the following command

javac -classpath `hadoop classpath` -d . MatrixMultiplicationDriver.java

STEP 11: Check the compile

ls

Must show :

MatrixMultiplicationDriver.class

STEP 12: Create JAR

*jar -cvf matrix.jar **

STEP 13: Run program

hadoop jar matrix.jar MatrixMultiplicationDriver /matrix /output

STEP 14: Output check

hdfs dfs -cat /output/part-r-00000

Output:

hadoop@boss:~$ hdfs dfs -cat /output/part-r-00000

0,0 6

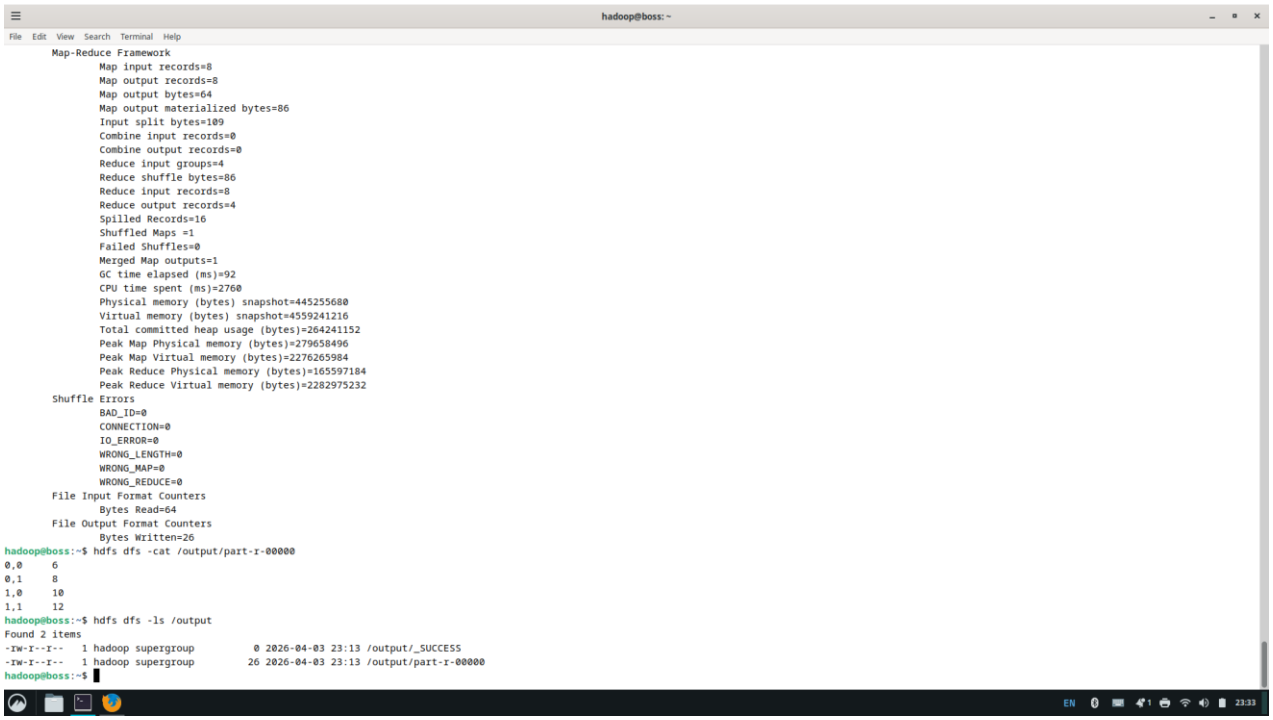
0,1 8

1,0 10

1,1 12

hadoop@boss:~$

```
hadoop@boss:~$ hadoop jar matrix.jar MatrixMultiplicationDriver /matrix /output
2026-04-03 23:12:28,633 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at boss/10.204.166.197:8032
2026-04-03 23:12:28,902 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-04-03 23:12:28,920 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1775224125940_0002
2026-04-03 23:12:37,360 INFO input.FileInputFormat: Total input files to process : 1
2026-04-03 23:12:37,463 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-03 23:12:37,920 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1775224125940_0002
2026-04-03 23:12:37,920 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-03 23:12:38,139 INFO conf.Configuration: resource-types.xml not found
2026-04-03 23:12:38,139 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-04-03 23:12:38,595 INFO impl.YarnClientImpl: Submitted application application_1775224125940_0002
2026-04-03 23:12:38,627 INFO mapreduce.Job: The url to track the job: http://boss:8088/proxy/application_1775224125940_0002/
2026-04-03 23:12:38,628 INFO mapreduce.Job: Running job: job_1775224125940_0002
2026-04-03 23:13:10,375 INFO mapreduce.Job: Job job_1775224125940_0002 running in uber mode : false
2026-04-03 23:13:10,377 INFO mapreduce.Job:  map 0% reduce 0%
2026-04-03 23:13:22,071 INFO mapreduce.Job:  map 100% reduce 0%
2026-04-03 23:13:34,315 INFO mapreduce.Job:  map 100% reduce 100%
2026-04-03 23:13:36,734 INFO mapreduce.Job: Job job_1775224125940_0002 completed successfully
2026-04-03 23:13:36,914 INFO mapreduce.Job: Counters: 54
```



RESULT:

Matrix multiplication was successfully implemented using Hadoop MapRed

EX.NO: 04	RUN A BASIC WORD COUNT MAP REDUCE PROGRAM
DATE:	

AIM:

To write and execute a basic Word Count program using Hadoop MapReduce to understand the MapReduce paradigm.

PROCEDURE:

The Word Count program in Hadoop MapReduce is used to count the frequency of each word present in a given input text file. The processing is done in three main phases: Mapper, Reducer and Driver.

STEP 1: MAPPER PHASE

The Mapper reads the input data line by line from the HDFS. Each line of the input file is treated as a value, and the key represents the byte offset of that line.

The Mapper processes each line and splits it into individual words using a tokenizer. For every word identified, the Mapper emits a key-value pair where the key is the word itself and the value is the integer 1.

This phase converts the input data into intermediate key-value pairs.

Example:

Input line:

Hello Hadoop Hello

Mapper Output:

Hello 1
Hadoop 1
Hello 1

STEP 2: SHUFFLE AND SORT PHASE

After the Mapper phase, Hadoop automatically performs the shuffle and sort operation. In this phase, all intermediate key-value pairs are grouped based on the key (word).

All occurrences of the same word are brought together and sent to the same Reducer.

Example:

Hello → [1, 1]
Hadoop → [1]

STEP 3: REDUCER PHASE

The Reducer receives the grouped key-value pairs from the shuffle phase. For each key (word), it iterates through the list of values and calculates the total sum.

The final output is produced as a key-value pair where the key is the word and the value is the total count of that word.

Example:

Hello → 2
Hadoop → 1

STEP 4: DRIVER PHASE

The Driver class is responsible for configuring and executing the MapReduce job. It sets the Mapper and Reducer classes, defines the data types for input and output, and specifies the input and output paths in HDFS.

The job is submitted to the Hadoop framework, which handles task scheduling, execution, and monitoring.

STEP 5: EXECUTION OF THE JOB

The program is compiled and packaged into a JAR file. The job is executed using the Hadoop command:

hadoop jar wordcount.jar WordCount /input /output

The output is stored in the specified HDFS output directory.

STEP 6: Create Java file

nano WordCount.java

PROGRAM :

```
import java.io.IOException;

import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class Map extends Mapper<LongWritable, Text, Text,
IntWritable> {
```

```
    public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
     String line = value.toString();
     StringTokenizer tokenizer = new StringTokenizer(line);
     while (tokenizer.hasMoreTokens()) {
      String word = tokenizer.nextToken();
      context.write(new Text(word), new IntWritable(1));
     }
    }
   }
   public static class Reduce extends Reducer<Text, IntWritable, Text,
IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
      throws IOException, InterruptedException {
     int sum = 0;
     for (IntWritable val : values) {
      sum += val.get();
     }
     context.write(key, new IntWritable(sum));
    }
   }
   public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Word Count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);
```

```
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

STEP 6.1: Compile the java program

javac -classpath `hadoop classpath` -d . WordCount.java

STEP 6.2: Create JAR

*jar -cvf wordcount.jar **

STEP 6.3: Check

ls

Should see:

Wordcount.jar

STEP 6.4: Create input file

nano input.txt

content:

Hello Hadoop Hello World
Hadoop is Big Data

STEP 7: verify file

cat input.txt

STEP 8: Create HDFS folder

hdfs dfs -mkdir /word

STEP 9: Upload file to HDFS

hdfs dfs -put input.txt /word

STEP 10: Check upload

hdfs dfs -ls /word

Should show:

Input.txt

STEP 6.4: Run the java program

hdfs dfs -rm -r /output
hadoop jar wordcount.jar WordCount /word /output

```
hadoop@boss:~$ hdfs dfs -rm -r /output
hadoop jar wordcount.jar WordCount /word /output
rm: `/output': No such file or directory
2026-04-04 00:01:38,194 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at boss/10.204.166.197:8032
2026-04-04 00:01:38,529 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-04-04 00:01:38,545 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1775224125940_0003
2026-04-04 00:01:55,035 INFO input.FileInputFormat: Total input files to process : 1
2026-04-04 00:01:55,742 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-04 00:01:56,511 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1775224125940_0003
2026-04-04 00:01:56,511 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-04 00:01:56,679 INFO conf.Configuration: resource-types.xml not found
2026-04-04 00:01:56,679 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-04-04 00:01:56,762 INFO impl.YarnClientImpl: Submitted application application_1775224125940_0003
2026-04-04 00:01:56,802 INFO mapreduce.Job: The url to track the job: http://boss:8088/proxy/application_1775224125940_0003/
2026-04-04 00:01:56,803 INFO mapreduce.Job: Running job: job_1775224125940_0003
2026-04-04 00:02:12,186 INFO mapreduce.Job: Job job_1775224125940_0003 running in uber mode : false
2026-04-04 00:02:12,188 INFO mapreduce.Job:  map 0% reduce 0%
2026-04-04 00:02:23,174 INFO mapreduce.Job:  map 100% reduce 0%
2026-04-04 00:02:34,501 INFO mapreduce.Job:  map 100% reduce 100%
2026-04-04 00:02:36,750 INFO mapreduce.Job: Job job_1775224125940_0003 completed successfully
2026-04-04 00:02:36,973 INFO mapreduce.Job: Counters: 54
```

```
                Reduce input records=8
                Reduce output records=6
                Spilled Records=16
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=86
                CPU time spent (ms)=2460
                Physical memory (bytes) snapshot=434360320
                Virtual memory (bytes) snapshot=4551520256
                Total committed heap usage (bytes)=258998272
                Peak Map Physical memory (bytes)=272994304
                Peak Map Virtual memory (bytes)=2277924864
                Peak Reduce Physical memory (bytes)=161366016
                Peak Reduce Virtual memory (bytes)=2273595392
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=44
        File Output Format Counters
                Bytes Written=43
hadoop@boss:~$ hdfs dfs -cat /output/part-r-00000
Big     1
Data    1
Hadoop  2
Hello   2
World   1
is      1
hadoop@boss:~$
```

RESULT :

The Word Count program was successfully executed and the frequency of words was obtained using Hadoop MapReduce

EX.NO: 05	CONFIGURATION OF HOMOGENEOUS MULTI-NODE HADOOP CLUSTER AND PERFORMING BASIC HDFS OPERATIONS
DATE:	

AIM

To install Hadoop, configure a homogeneous multi-node cluster, and perform basic HDFS operations.

PREREQUISITES

- Java installed
- Linux system (Ubuntu)
- Two or more machines connected in network
- SSH enabled

PROCEDURE:

PART A :HADOOP INSTALLATION (MASTER & SLAVE)

STEP 1: Install Java

sudo apt update
sudo apt install openjdk-8-jdk -y

STEP 1.1: Set JAVA_HOME

nano ~/.bashrc

Add:

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/binsource ~/.bashrc

STEP 2: Create Hadoop User

sudo adduser hadoop
sudo usermod -aG sudo hadoop
su - hadoop

STEP 3: Install SSH

sudo apt install openssh-server -y

STEP 4: Download Hadoop

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

STEP 5: Extract Hadoop

tar -xvzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop

STEP 6: Set Hadoop Environment

nano ~/.bashrc

Add:

export HADOOP_HOME=/home/hadoop/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbinsource ~/.bashrc

STEP 7: Configure SSH (Passwordless)

ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

```
chmod 600 ~/.ssh/authorized_keys
ssh localhost
```

PART B: MULTI-NODE CONFIGURATION

STEP 8: Configure Hosts (Both Systems)

```
sudo nano /etc/hosts
```

Add:

```
192.168.1.10 master
192.168.1.11 slave
```

STEP 9: Copy SSH Key to Slave

```
ssh-copy-id hadoop@slave
```

STEP 10: Copy Hadoop to Slave

```
scp -r ~/hadoop hadoop@slave:/home/hadoop/
```

STEP 11: Hadoop Configuration (Master)

```
cd ~/hadoop/etc/hadoop
```

core-site.xml

```
<configuration>
 <property>
  <name>fs.defaultFS</name>
  <value>hdfs://master:9000</value>
 </property>
</configuration>
```

hdfs-site.xml

```
<configuration>
 <property>
  <name>dfs.replication</name>
  <value>2</value>
```

</property>
</configuration>

mapred-site.xml

cp mapred-site.xml.template mapred-site.xml
nano mapred-site.xml<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>

yarn-site.xml

nano yarn-site.xml<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>

STEP 12: Workers File

nano $HADOOP_HOME/etc/hadoop/workers

Add:

slave

STEP 13: Sync Config to Slave

scp -r ~/hadoop/etc/hadoop hadoop@slave:/home/hadoop/hadoop/etc/

PART C: EXECUTION

STEP 14: Format NameNode

hdfs namenode -format

STEP 15: Start Hadoop

start-dfs.sh
start-yarn.sh

STEP 16: Verify

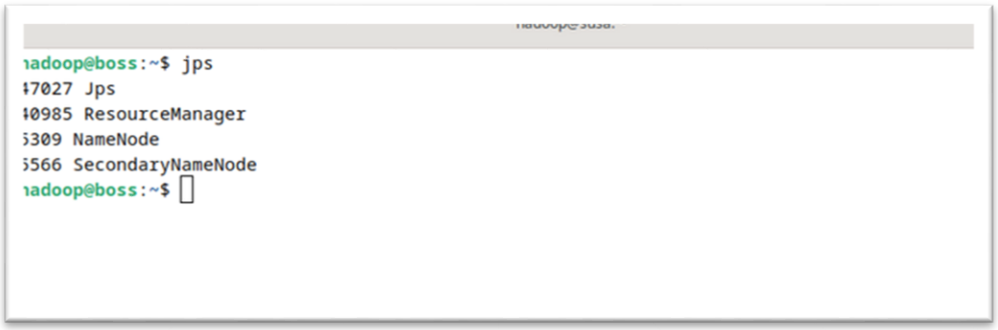
Jps

STEP 17: Test HDFS

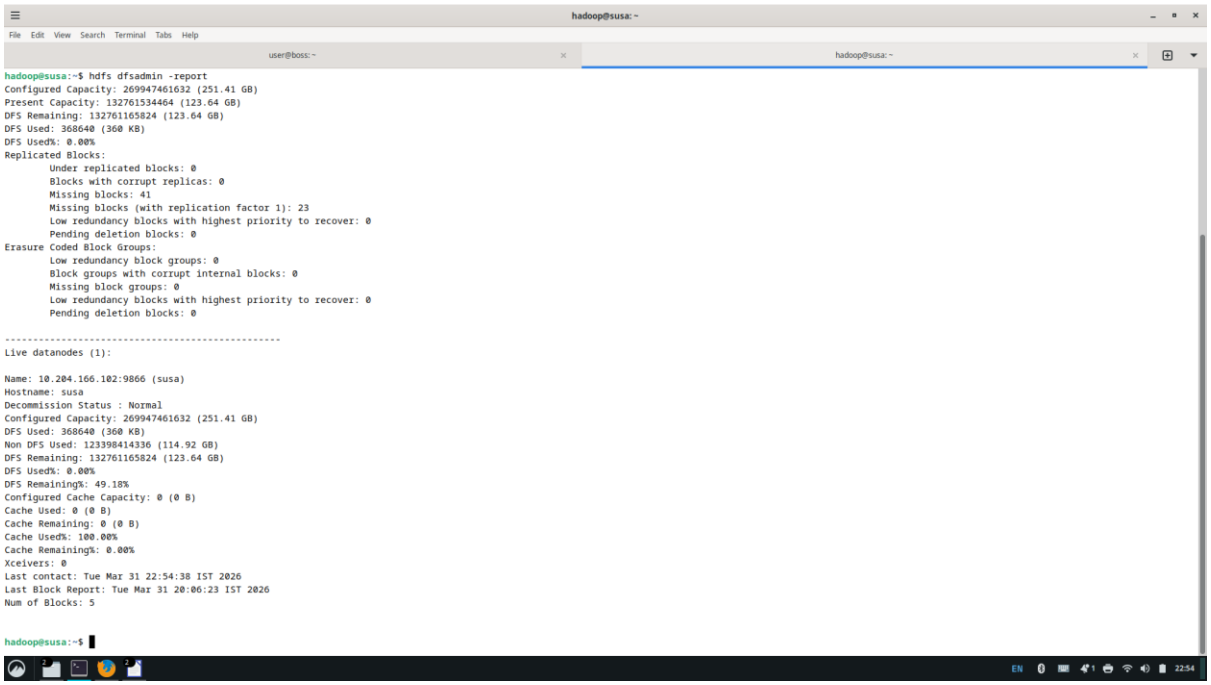
echo "Cluster Working" > test.txt
hdfs dfs -mkdir /test
hdfs dfs -put test.txt /test
hdfs dfs -cat /test/test.txt

OUTPUT:

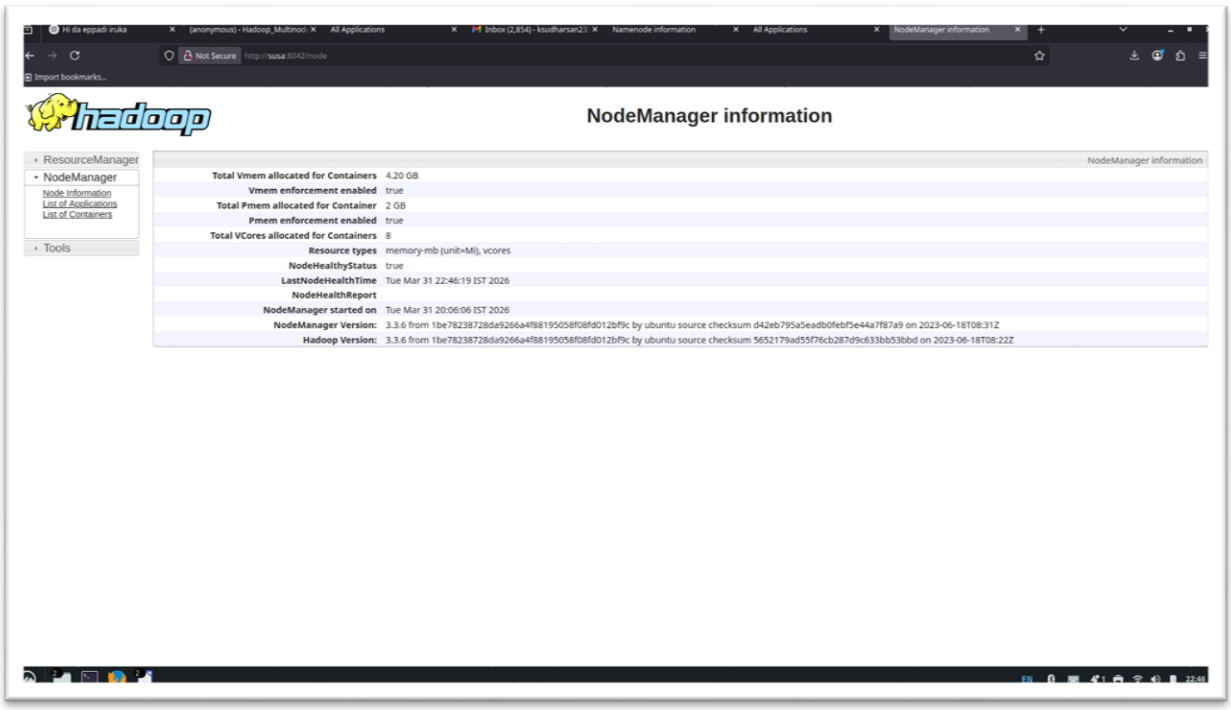
Cluster Working



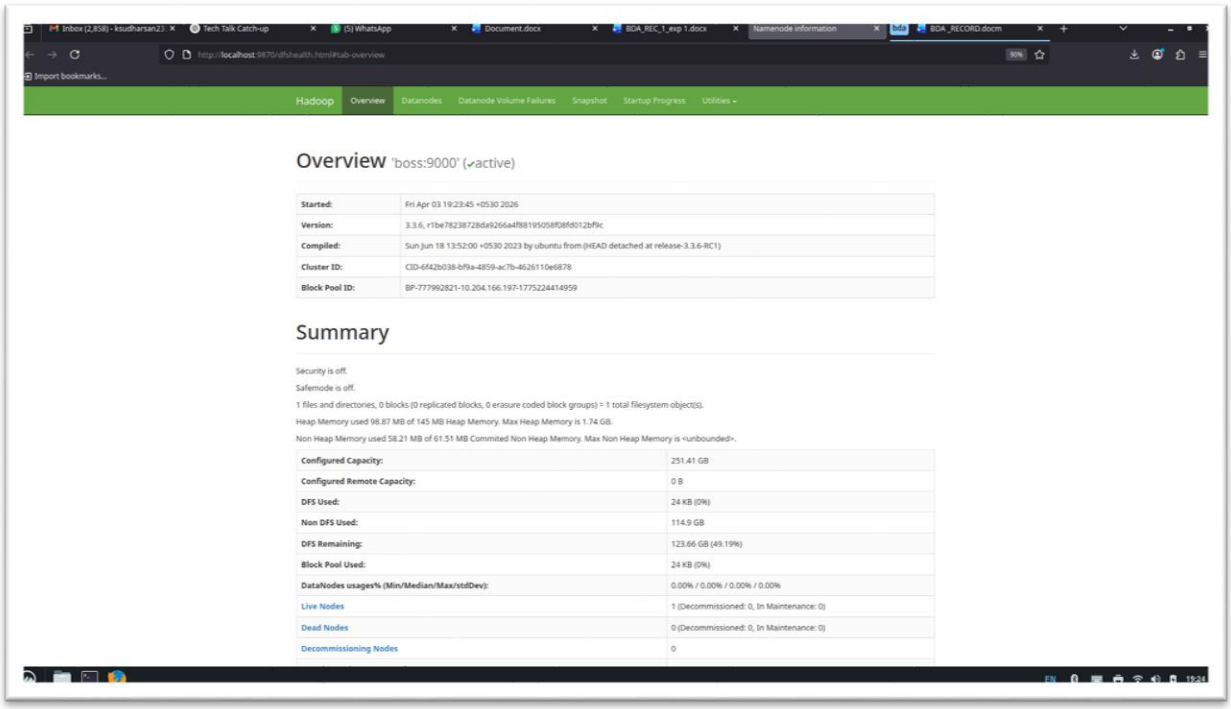
JPS



hdfs admin report



ResourceManager UI: http://master:8088



NameNode UI: http://master:9870

RESULT:

Thus, Hadoop was successfully installed and a homogeneous multi-node cluster was configured. HDFS operations were performed successfully.

EX.NO: 06	HADOOP INSTALLATION AND HETEROGENEOUS MULTI-NODE CLUSTER CONFIGURATION
DATE:	

AIM:

To install Hadoop, configure a heterogeneous multi-node cluster, and perform basic HDFS operations.

PREREQUISITES:

- Java installed (same version across all nodes)
- Linux system (Ubuntu or other distributions)
- Two or more machines connected in network
- SSH enabled
- Hadoop same version in all nodes

PROCEDURE:

PART A : HADOOP INSTALLATION (MASTER & SLAVE)

STEP 1: Install Java

sudo apt update

sudo apt install openjdk-8-jdk -y

STEP 1.1: Set JAVA_HOME

nano ~/.bashrc

Add:

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

export PATH=$PATH:$JAVA_HOME/bin

Apply :

source ~/.bashrc

STEP 2: Create Hadoop User

sudo adduser hadoop

sudo usermod -aG sudo hadoop

su - hadoop

STEP 3: Install SSH

sudo apt install openssh-server –y

STEP 4: Download Hadoop

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

STEP 5: Extract Hadoop

tar -xvzf hadoop-3.3.6.tar.gz

mv hadoop-3.3.6 hadoop

STEP 6: Set Hadoop Environment

nano ~/.bashrc

Add:

export HADOOP_HOME=/home/hadoop/hadoop

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

Apply:

source ~/.bashrc

STEP 7: Configure SSH (Passwordless)

ssh-keygen -t rsa

cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys

ssh localhost

PART B : MULTI-NODE CONFIGURATION

STEP 8: Configure Hosts (Both Systems)

sudo nano /etc/hosts

Add:

192.168.1.10 master

192.168.1.11 slave

STEP 9: Copy SSH Key to Slave

ssh-copy-id hadoop@slave

STEP 10: Copy Hadoop to Slave

scp -r ~/hadoop hadoop@slave:/home/hadoop/

STEP 11: Hadoop Configuration (MASTER)

cd ~/hadoop/etc/hadoop

core-site.xml

nano core-site.xml

```
<configuration>
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://master:9000</value>
</property>
</configuration>
```

hdfs-site.xml

nano hdfs-site.xml

```
<configuration>
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
</configuration>
```

mapred-site.xml

cp mapred-site.xml.template mapred-site.xml

nano mapred-site.xml

```
<configuration>
<property>
 <name>mapreduce.framework.name</name>
 <value>yarn</value>
</property>
</configuration>
```

yarn-site.xml

nano yarn-site.xml

```
<configuration>
<property>
 <name>yarn.nodemanager.resource.memory-mb</name>
 <value>2048</value>
</property>
<property>
 <name>yarn.scheduler.maximum-allocation-mb</name>
 <value>1024</value>
</property>
<property>
 <name>yarn.nodemanager.aux-services</name>
 <value>mapreduce_shuffle</value>
</property>
</configuration>
```

STEP 12: Workers File

nano $HADOOP_HOME/etc/hadoop/workers

Add:

slave

STEP 13: Sync Config to Slave

scp -r ~/hadoop/etc/hadoop hadoop@slave:/home/hadoop/hadoop/etc/

STEP 14: Network Verification

ping slave

ssh slave

PART C : EXECUTION

STEP 15: Format NameNode

hdfs namenode -format

STEP 16: Start Hadoop only in master node

start-dfs.sh

start-yarn.sh

STEP 17: Verify both machine (master & slave)

Jps

Must show:

MASTER NODE:

NameNode

ResourceManager

SecondaryNameNode

SLAVE NODE:

DataNode

NodeManager

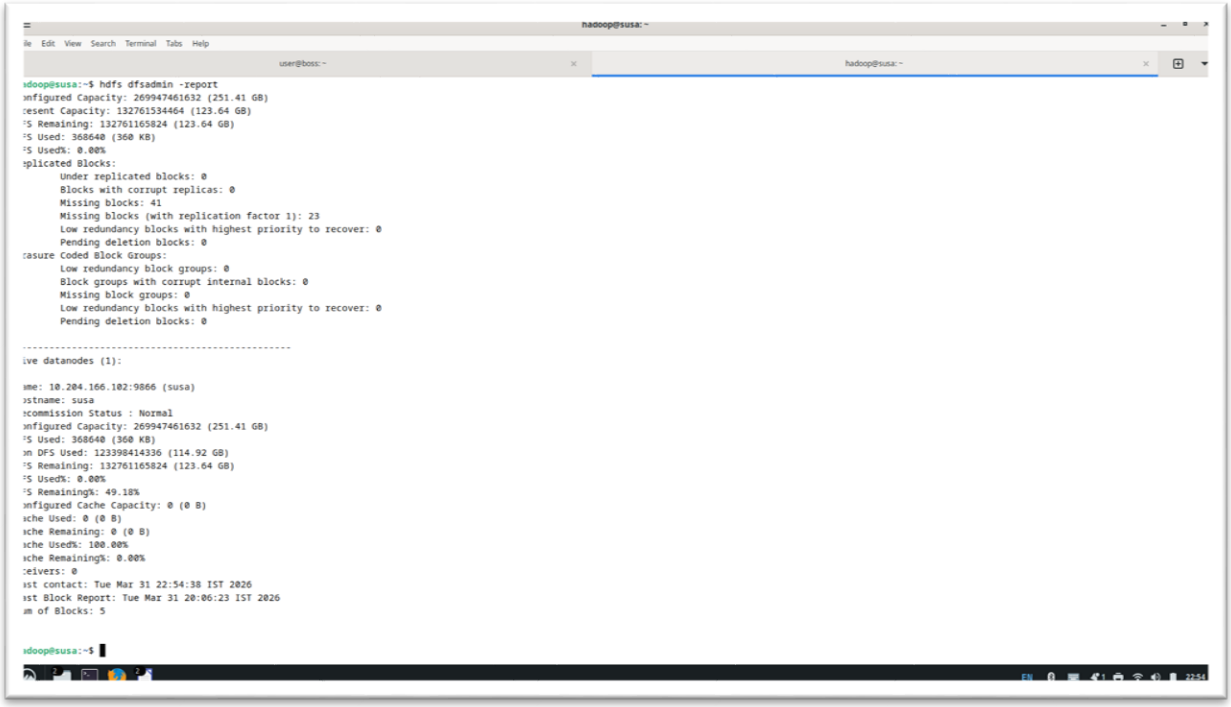
STEP 18: Test HDFS

```
echo "Cluster Working" > test.txt
hdfs dfs -mkdir /test
hdfs dfs -put test.txt /test
hdfs dfs -cat /test/test.txt
```

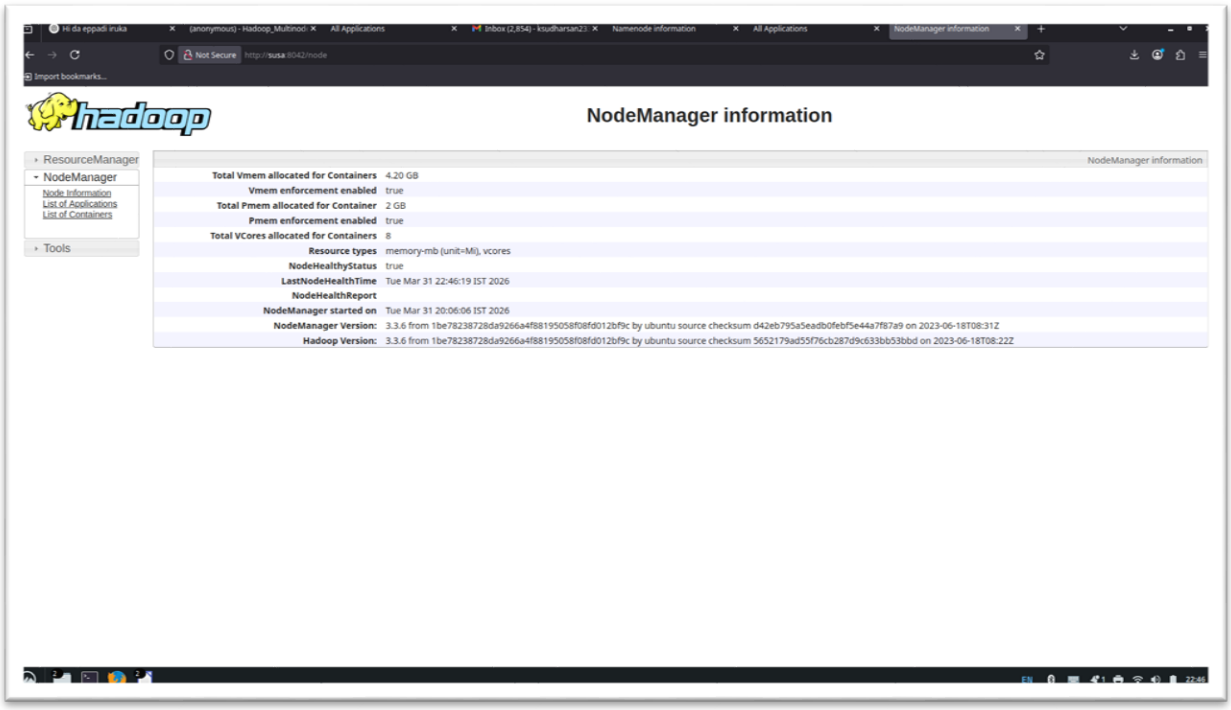
OUTPUT:

Cluster Working

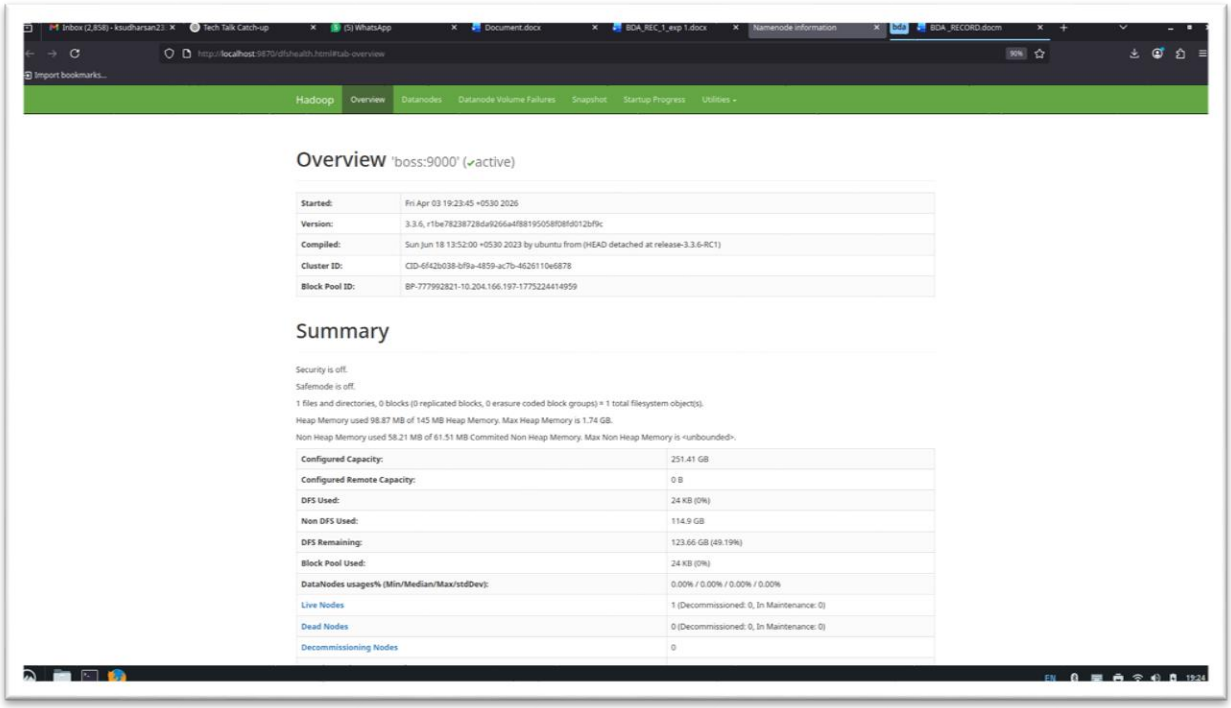
ADDITIONAL VERIFICATION



hdfs dfsadmin –report



ResourceManager UI:http://master:8088



NameNode UI:http://master:9870

RESULT:

Thus, Hadoop was successfully installed and a heterogeneous multi-node cluster was configured. HDFS operations were performed successfully.

EX.NO: 07	

DATE:	INSTALLATION OF HIVE ALONG WITH PRACTICE EXAMPLES

AIM:

To install Apache Hive and execute basic queries using Hive.

PREREQUISITES:

- Java Development Kit (JDK) installed and JAVA_HOME set
- Hadoop installed and configured
- HDFS running

DESCRIPTION:

Apache Hive:

Apache Hive is an open-source data warehouse system built on top of Apache Hadoop, designed for querying and analyzing large datasets stored in HDFS. It provides a SQL-like query language called HiveQL, which allows users to perform data analysis without writing complex MapReduce programs.

Hive translates HiveQL queries into underlying execution frameworks such as MapReduce, Tez, or Spark, enabling efficient processing of big data. It is mainly used for batch processing and data warehousing tasks.

Hive organizes data into databases, tables, and partitions, similar to traditional relational databases, but it is optimized for handling large-scale distributed data. It supports structured data and provides features like schema-on-read, meaning the schema is applied when data is read rather than when it is stored.

In this experiment, Apache Hive is installed and configured, and basic operations such as creating tables, inserting data, querying data, and performing aggregations are carried out to understand how Hive processes big data using SQL-like commands.

PROCEDURE:

STEP 1: Download Hive

wget https://archive.apache.org/dist/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz

STEP 2 : Extract the downloaded Hive

tar -xzf apache-hive-3.1.3-bin.tar.gz

STEP 3: Move Hive to Directory

sudo mv apache-hive-3.1.3-bin /usr/local/hive

STEP 4: Environment Variables Setup

nano ~/.bashrc

Add:

export HIVE_HOME=/usr/local/hive
export PATH=$PATH:$HIVE_HOME/bin

Apply:

source ~/.bashrc

STEP 5: Configure Hive

cd $HIVE_HOME/conf

cp hive-default.xml.template hive-site.xml

Edit:

nano hive-site.xml

Add:

<configuration>
<property>
<name>javax.jdo.option.ConnectionURL</name
<value>jdbc:derby:;databaseName=/home/hadoop/metastore_db;create=true</value>
</property>
</configuration>

STEP 6: Initialize Metastore

schematool -dbType derby –initSchema

STEP 7: Start Hadoop

start-dfs.sh
start-yarn.sh

S**TEP 8:** Start Hive

Command:

hive

See in terminal :

hive>

STEP 9: Set Local Mode:

SET hive.exec.mode.local.auto=true;

STEP 10: Test queries

CREATE DATABASE mydb;

USE mydb;

CREATE TABLE student(

id INT,

name STRING,

age INT

);

INSERT INTO student VALUES (1,'appu,22),(2,'dinesh',20);

SELECT * FROM student;

Output:

hive> SELECT * FROM student;

OK

1 Appu 22

2 Dinesh 20

Time taken: 1.929 seconds, Fetched: 2 row(s)

hive>

```
hadoop@boss:~$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = e9e04442-8ae5-4d5b-828e-5fdd06b88a70

Logging initialized using configuration in jar:file:/usr/local/hive/lib/hive-common-3.1.3.jar!/hive-log4j2.properties Async: true
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Hive Session ID = 812ad7d2-c393-48cc-bddc-00989603b3fc
hive> SELECT * FROM student;
OK
1       Appu    22
2       Dinesh  20
Time taken: 1.929 seconds, Fetched: 2 row(s)
hive>
```

RESULT:

Thus, Apache Hive was successfully installed and various queries such as database creation, table creation, data insertion, and data retrieval were executed successfully.

EX.NO: 08	**INSTALLATION OF HBASE ALONG WITH PRACTICE EXAMPLES**
DATE:	

AIM:

To install Apache HBase in Linux environment and perform basic operations in HBase.

PREREQUISITES:

- Java installed
- Hadoop installed and running
- HDFS working

DESCRIPTION:

Apache HBase:

Apache HBase is an open-source, distributed NoSQL database built on top of Apache Hadoop and runs on the Hadoop Distributed File System (HDFS). It is designed to store and manage large volumes of structured and semi-structured data across distributed systems.

HBase follows a column-oriented storage model, where data is stored in tables consisting of rows and column families. It provides real-time read and write access to data, making it suitable for applications that require fast and random access to big data.

HBase is highly scalable and can handle billions of rows and millions of columns efficiently. It supports automatic sharding through regions and ensures fault tolerance using HDFS replication.

In this experiment, HBase is installed and configured to perform basic operations such as creating tables, inserting data, retrieving data, and scanning records. These operations help in understanding how HBase manages and processes large-scale data in a distributed environment.

PROCEDURE:

STEP 1: Install Required Software

- Install Linux (Ubuntu/BOSS OS) in Virtual Machine
- Ensure network connectivity
- Install Java and Hadoop

STEP 2: Install Java

sudo apt update
sudo apt install default-jdk -y
java -version

STEP 3: Download HBase

wg*et https://archive.apache.org/dist/hbase/2.4.17/hbase-2.4.17-bin.tar.gz*

STEP 4: Extract HBase

tar -xvf hbase-2.4.17-bin.tar.gz

STEP 5: Move HBase

sudo mv hbase-2.4.17 /usr/local/hbase

STEP 6: Configure HBase

cd /usr/local/hbase/conf
nano hbase-site.xml

Add:

```
<configuration>
<property>
<name>hbase.rootdir</name>
 <value>file:///home/hadoop/hbase-data</value>
</property>

 <property>
 <name>hbase.zookeeper.property.dataDir</name>
<value>/home/hadoop/zookeeper</value>
</property>
</configuration>
```

STEP 7: Set JAVA_HOME

nano /usr/local/hbase/conf/hbase-env.sh

Add:

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

STEP 8: Start HBase

/usr/local/hbase/bin/start-hbase.sh

STEP 9: Check
Command : *jps*
Show:

```
hadoop@boss:~$ jps
        5440 HRegionServer
        5104 HQuorumPeer
        5233 HMaster
        4357 ResourceManager
        3864 NameNode
```

5866 Jps
4125 SecondaryNameNode

STEP 10: Open HBase Shell

Command: hbase shell

STEP 11: Check status

Command : status

Now should be:

1 active master, 1 server, 0 dead

STEP 12: Now create table

Command: *create 'my_table', 'cf'*

```
hadoop@boss:~$ hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hbase/lib/client-facing-thirdparty/slf4j-reload4j-1.7.33.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Reload4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.17, r7fd096f39b4284da9a71da3ce67c48d259ffa79a, Fri Mar 31 18:10:45 UTC 2023
Took 0.0022 seconds
hbase:001:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
Took 0.5401 seconds
hbase:002:0> create 'my_table', 'cf'
Created table my_table
Took 0.7325 seconds
=> Hbase::Table - my_table
hbase:003:0>
```

STEP 13 : To insert data into the table, use the put command.

put 'my_table', 'row1', 'cf:name', 'Appu'

put 'my_table', 'row2', 'cf:name', 'Dinesh'

STEP 14: view the created table

scan 'my_table'

```
=> Hbase::Table - my_table
hbase:003:0> put 'my_table', 'row1', 'cf:name', 'Appu'
Took 0.1789 seconds
hbase:004:0> put 'my_table', 'row2', 'cf:name', 'Dinesh'
Took 0.0077 seconds
hbase:005:0> scan 'my_table'
ROW                                             COLUMN+CELL
 row1                                           column=cf:name, timestamp=2026-04-04T21:17:08.585, value=Appu
 row2                                           column=cf:name, timestamp=2026-04-04T21:17:12.240, value=Dinesh
2 row(s)
Took 0.0387 seconds
hbase:006:0>
```

STEP 15: To retrieve data of a specific row, use the get command.

get 'my_table', 'row1'

STEP 16: To delete a specific value:

delete 'my_table', 'row1', 'cf:name'

This deletes column value of row1.

STEP 17: To delete the entire table:

disable 'my_table'

drop 'my_table'

```
hbase:007:0> get 'my_table', 'row1'
COLUMN                                          CELL
 cf:name                                        timestamp=2026-04-04T21:17:08.585, value=Appu
1 row(s)
Took 0.0221 seconds
hbase:008:0> delete 'my_table', 'row1', 'cf:name'
Took 0.0091 seconds
hbase:009:0> disable 'my_table'
Took 0.3869 seconds
hbase:010:0> drop 'my_table'
Took 0.3713 seconds
hbase:011:0>
```

RESULT:

Thus, Apache HBase was successfully installed and operations such as table creation, data insertion, retrieval, and deletion were performed successfully.

EX.NO: 09	INSTALLATION OF THRIFT ALONG WITH PRACTICE EXAMPLES
DATE:	

AIM:

To install Apache Thrift and connect it with Apache HBase to perform basic operations.

PREREQUISITES:

- Java installed
- Hadoop installed and running
- HBase installed and running
- Linux (Ubuntu / WSL)

DESCRIPTION:

Apache Thrift:

Apache Thrift is an open-source framework developed by Apache Software Foundation for building scalable and cross-language services. It allows communication between applications written in different programming languages by defining data types and service interfaces using a common interface definition language (IDL).

Thrift provides a combination of a software stack and code generation engine to enable efficient Remote Procedure Calls (RPC). It supports multiple programming languages such as Java, Python, C++, and more, making it highly flexible for distributed systems.

In the context of Apache HBase, Apache Thrift acts as a bridge that allows external applications (like Python programs) to interact with HBase. By running the Thrift server,

clients can perform operations such as inserting, retrieving, and updating data in HBase tables using simple APIs.

Thus, Apache Thrift enables seamless communication between HBase and external applications, making it useful for building distributed and language-independent big data applications.

PROCEDURE:

STEP 1: Start Hadoop Services

start-dfs.sh
start-yarn.sh

STEP 2: Start HBase

/usr/local/hbase/bin/start-hbase.sh

STEP 3: Install Thrift

sudo apt update
sudo apt install thrift-compiler -y

Check version:

thrift -version

STEP 4: Navigate to HBase Directory

cd /usr/local/hbase

STEP 5: Start Thrift Server

/bin/hbase-daemon.sh start thrift

OR

hbase-daemon.sh start thrift

STEP 6: Verify Thrift is Running

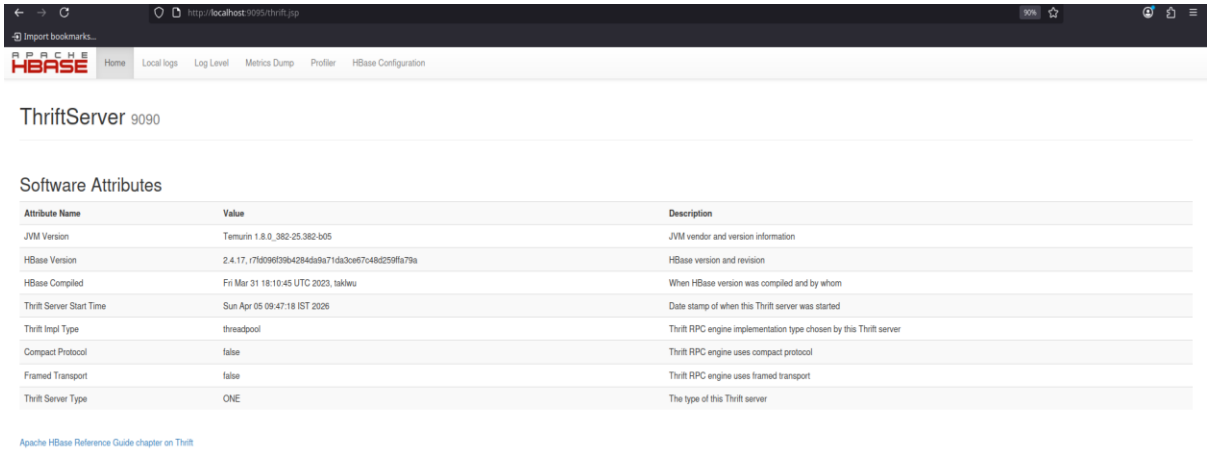
jps

Output should show:

ThriftServer
HMaster
HRegionServer

STEP 7: Open HBase Shell

hbase shell



PYTHON + THRIFT SETUP

STEP 1: Python check pannunga

python3 --version

Output

Python 3.11.2

STEP 1.1: If python is not installed , install the python

sudo apt update

sudo apt install python3 –y

STEP 2: pip install

sudo apt install python3-pip –y

Check pip

pip3 –version

STEP 3: Create aVirtual environment (venv)

```
python3 -m venv myenv
```

STEP 4: Activate the created virtual environment

```
source myenv/bin/activate
Output:
(myenv) hadoop@boss:~$
```

STEP 5: Required packages install

```
pip install happybase thrift
```

STEP 6: Create a Python file

```
nano test.py
```

STEP 7: Add this code

```
import happybase
print("Connecting...")
connection = happybase.Connection('localhost', 9090)
table = connection.table('emp')
table.put(b'1', {b'info:name': b'Ramana', b'info:age': b'21'})
print("Data inserted successfully")
```

STEP 8: Execute the python code

```
python3 test.py
```

Output:

```
Data inserted successfully
```

STEP 9: Verify in HBase

```
hbase shell
(myenv) hadoop@boss:~$ hbase shell
Step 10: Run inside the HBase shell
scan 'emp'
hbase:001:0> scan 'emp'
ROW                    COLUMN+CELL
1          column=info:age, timestamp=2026-04-05T09:02:51.723,  value=25
```

1 column=info:name, timestamp=2026-04-05T09:02:50.619, value=Car

2 column=info:age, timestamp=2026-04-05T10:16:07.508, value=21

2 column=info:name, timestamp=2026-04-05T10:16:07.508, value=Ramana

2 row(s)

Took 0.3952 seconds

STEP 11: Update the python file

Add:

```
import happybase
connection = happybase.Connection('localhost', 9090)
table = connection.table('emp')
table.put(b'3', {b'info:name': b'Charu', b'info:age': b'21'})
table.put(b'4', {b'info:name': b'Raji', b'info:age': b'20'})
print("Multiple records inserted")
```

STEP 12: Run the python file

```
python3 test.py
```

Output:

(myenv) hadoop@boss:~$ python3 test.py

Multiple records inserted

STEP 13: check in HBase shell

```
scan 'emp'
```

OUTPUT:

```
hbase:001:0> scan 'emp'
ROW                                                   COLUMN+CELL
 1                                                    column=info:age, timestamp=2026-04-05T09:02:51.723, value=25
 1                                                    column=info:name, timestamp=2026-04-05T09:02:50.619, value=Carn
 2                                                    column=info:age, timestamp=2026-04-05T10:16:07.508, value=21
 2                                                    column=info:name, timestamp=2026-04-05T10:16:07.508, value=Ramana
 3                                                    column=info:age, timestamp=2026-04-05T10:20:03.456, value=21
 3                                                    column=info:name, timestamp=2026-04-05T10:20:03.456, value=Charu
 4                                                    column=info:age, timestamp=2026-04-05T10:20:03.461, value=20
 4                                                    column=info:name, timestamp=2026-04-05T10:20:03.461, value=Raji
4 row(s)
Took 0.3862 seconds
hbase:002:0>
```

RESULT:

Thus, **Apache Thrift was installed and integrated with HBase**, and basic operations were performed successfully.

EX.NO: 10	**INTEGRATION OF MONGODB, APACHE SPARK, AND HADOOP (HDFS) FOR DATA IMPORT AND EXPORT OPERATIONS**
DATE:	

AIM:

To perform data import from MongoDB to HDFS and export back from HDFS to MongoDB using Apache Spark.

REQUIREMENTS:

- Hadoop (HDFS)
- Apache Spark
- MongoDB
- Java (JDK 8)
- Linux System

DESCRIPTION:

MongoDB:

MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It is schema-less, highly scalable, and designed to handle large volumes of unstructured data. MongoDB supports high performance, horizontal scaling, and real-time data processing, making it suitable for big data applications.

Apache Spark:

Apache Spark is an open-source distributed data processing framework used for large-scale data analytics. It provides high-speed processing through **in-memory computation**, making it faster than traditional Hadoop MapReduce.

Spark uses a master-worker architecture where the driver program coordinates tasks and worker nodes process data in parallel. It supports multiple languages like Scala, Python, and Java, and provides components such as Spark SQL and MLlib for advanced data processing.

It can integrate with various data sources like **HDFS, MongoDB, and relational databases**, making it suitable for ETL operations and big data pipelines. In this experiment, Spark acts as a bridge between MongoDB and Hadoop for efficient data transfer.

ARCHITECTURE:

MongoDB → Spark → HDFS (Import)

HDFS → Spark → MongoDB (Export)

PROCEDURE:

STEP 1: Start Hadoop Services

start-dfs.sh
start-yarn.sh

STEP 2: Verify Hadoop

jps

STEP 3: start the MongoDB

mongosh

OUTPUT:

jolly@boss:~$ mongosh

Current Mongosh Log ID: 69d7bbd3d96fdd922444ba88

Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2

Using MongoDB: 6.0.27

Using Mongosh: 2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

The server generated these startup warnings when booting

2026-04-09T19:55:35.027+05:30: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem

2026-04-09T19:55:35.786+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

2026-04-09T19:55:35.786+05:30: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never' in this binary version

2026-04-09T19:55:35.786+05:30: vm.max_map_count is too low

---

test>

Step 3.1: Insert the data

```
use testdb
db.students.insertMany([
 {id:1, name:"Appu", age:21},
 {id:2, name:"Dinesh", age:22},
```

```
 {id:3, name:"Ramana", age:23}
])
```

STEP 4: Start Spark with Mongo Connector

```
spark-shell --packages org.mongodb.spark:mongo-spark-connector_2.12:3.0.1
```

STEP 5: Read Data from MongoDB

```
val df = spark.read.format("mongo").option("uri","mongodb://127.0.0.1/
testdb.students").load()
df.show()
```

Output:

```
scala> df.show()
+--------------------+---+----+------------------+
|                 _id |      age|     id|     name |
+--------------------+---+----+------------------+
|{69d69ae9ec263bcc...| 21|NULL|          car|
|{69d7bdcad96fdd92...| 21|   1     |      Appu|
|{69d7bdcad96fdd92...| 22|   2     |    Dinesh|
|{69d7bdcad96fdd92...| 23|   3     |   Ramana|
+------------------------+---+-------+----------+
```

STEP 6: Write Data to HDFS (Import)

```
df.write.mode("overwrite").json("hdfs://localhost:9000/mongo-data")
```

STEP 7: Read Data from HDFS

```
val df2 = spark.read.json("hdfs://localhost:9000/mongo-data")
df2.show()
```

STEP 8: Write Data back to MongoDB (Export)

```
df2.write.format("mongo").option("uri","mongodb://127.0.0.1/
testdb.final_output").mode("overwrite").save()
```

STEP 9: Verify Output in MongoDB

```
use testdb
show collections
db.final_output.find()
```

```
testdb> show collections
final_output
students
testdb> db.final_output.find()
[
  {
    _id: ObjectId('69d69ae9ec263bcc9744ba89'),
    age: Long('21'),
    name: 'car'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba89'),
    age: Long('21'),
    id: Long('1'),
    name: 'Appu'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba8a'),
    age: Long('22'),
    id: Long('2'),
    name: 'Dinesh'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba8b'),
    age: Long('23'),
    id: Long('3'),
    name: 'Ramana'
  }
]
testdb>
```

OUTPUT: **IMPORT THE DATA**

```
cala> val df = spark.read.format("mongo").option("uri","mongodb://127.0.0.1/testdb.students").load()
f: org.apache.spark.sql.DataFrame = [_id: struct<oid: string>, age: int ... 2 more fields]

cala> df.write.mode("overwrite").json("hdfs://localhost:9000/mongo-data")

cala> df.show()
--------------------+---+----+------+
                 _id|age|  id|  name|
--------------------+---+----+------+
{69d69ae9ec263bcc...| 21|NULL|   car|
{69d7bdcad96fdd92...| 21|   1|  Appu|
{69d7bdcad96fdd92...| 22|   2|Dinesh|
{69d7bdcad96fdd92...| 23|   3|Ramana|
--------------------+---+----+------+


cala>
```

SPARK

```
sword:
ly@boss:~$ hdfs dfs -ls /mongo-data
nd 2 items
-r--r--   3 jolly supergroup          0 2026-04-09 22:46 /mongo-data/_SUCCESS
-r--r--   3 jolly supergroup        288 2026-04-09 22:46 /mongo-data/part-00000-c4a3e4c4-7021-4b57-a1e3-e01e555223b9-c000.json
ly@boss:~$ hdfs dfs -cat /mongo-data/*
_id":{"oid":"69d69ae9ec263bcc9744ba89"},"age":21,"name":"car"}
_id":{"oid":"69d7bdcad96fdd922444ba89"},"age":21,"id":1,"name":"Appu"}
_id":{"oid":"69d7bdcad96fdd922444ba8a"},"age":22,"id":2,"name":"Dinesh"}
_id":{"oid":"69d7bdcad96fdd922444ba8b"},"age":23,"id":3,"name":"Ramana"}
ly@boss:~$
```

HDFS

OUTPUT: **EXPORT THE DATA**

```
scala> val df2 = spark.read.json("hdfs://localhost:9000/mongo-data"); df2.show()
+--------------------+---+----+------+
|                 _id|age|  id|  name|
+--------------------+---+----+------+
|{69d69ae9ec263bcc...| 21|NULL|   car|
|{69d7bdcad96fdd92...| 21|   1|  Appu|
|{69d7bdcad96fdd92...| 22|   2|Dinesh|
|{69d7bdcad96fdd92...| 23|   3|Ramana|
+--------------------+---+----+------+

df2: org.apache.spark.sql.DataFrame = [_id: struct<oid: string>, age: bigint ... 2 more fields]

scala> df2.write.format("mongo").option("uri","mongodb://127.0.0.1/testdb.final_output").mode("overwrite").save()

scala>
```

SPARK

```
testdb> show collections
final_output
students
testdb> db.final_output.find()
[
  {
    _id: ObjectId('69d69ae9ec263bcc9744ba89'),
    age: Long('21'),
    name: 'car'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba89'),
    age: Long('21'),
    id: Long('1'),
    name: 'Appu'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba8a'),
    age: Long('22'),
    id: Long('2'),
    name: 'Dinesh'
  },
  {
    _id: ObjectId('69d7bdcad96fdd922444ba8b'),
    age: Long('23'),
    id: Long('3'),
    name: 'Ramana'
  }
]
testdb>
```

MONGODB

RESULT:

The data was successfully transferred from MongoDB to HDFS and back to MongoDB using Apache Spark.